Character Mapping Edges

Alex Beams

Department of Mathematics
Simon Fraser University

PhylogeogRaphy Workshop

Day 2

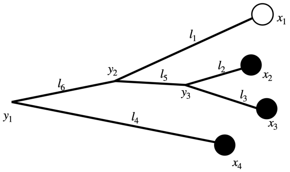
Table of Contents

Summarizing up to now:

Defining Markov models on trees allows us to model trait changes over time.

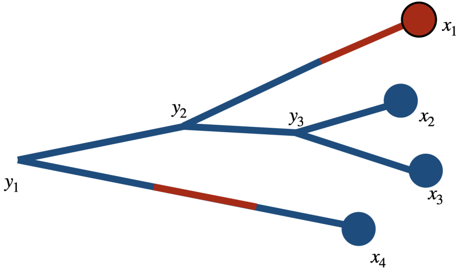
The likelihood function $\sum_{\mathbf{y}} p(\mathbf{x}, \mathbf{y} | \mathcal{T})$ is calculated using transition probabilities of a Markov chain, and is a marginalization over unobserved ancestral states, $\mathbf{y}$

After fitting a model with Maximum Likelihood, we can estimate ancestral states $\mathbf{y}$ through a joint reconstruction, or individual states y_k through a marginal reconstruction; these may or may not be compatible with each other (but this isn't a bug – joint and marginal reconstruction address fundamentally different questions).



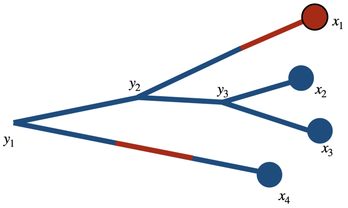
STOCHASTIC CHARACTER MAPPING

In addition to estimating ancestral states at particular nodes, we may wish to reconstruct state changes along branches in a phylogeny:



Stochastic character mapping **augments** ancestral state reconstruction at nodes.

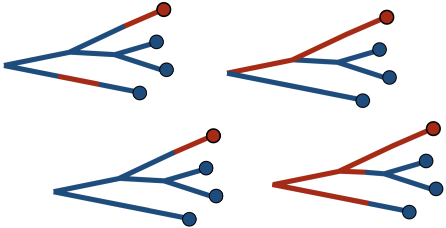
Stochastic character mapping



This proceeds in two (actually, three) steps:

0 Fit a Markov model (using ace from ape, or FitMk from phytools) to obtain a generator, **Q**
1 Use the Markov model to sample ancestral states at internal nodes (should be a joint reconstruction)
2 Simulate changes of state along each branch, ensuring that boundary conditions at nodes are satisfied

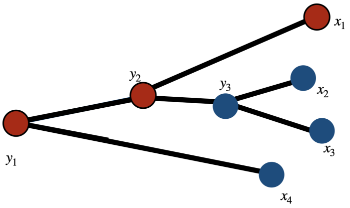
Sampling node configurations



It is typical to simulate character mappings to get a sense of the distribution of possible evolutionary histories, rather than the most likely evolutionary history.

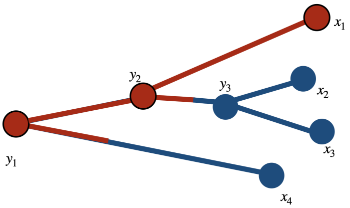
Simulate state changes along branches

This requires first sampling joint configurations of ancestral states at internal nodes. We can easily modify the joint reconstruction algorithm we saw previously to do this



Simulate state changes along branches

Once an internal node configuration has been sampled through a marginal or joint approach, simulate the Markov chain along each branch



Stochastic mapping vs. ancestral character estimation

Stochastic mapping is not a replacement for ancestral character estimation – it augments it

Need to do all of the ancestral character estimation calculations to do stochastic character mapping

Stochastic character mapping helps us count the number of trait changes in a phylogeny. Ancestral character estimation does not directly give this to us

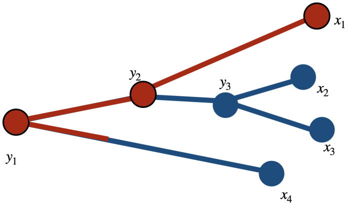


Table of Contents

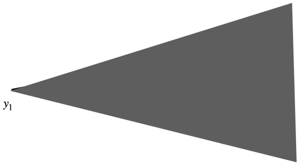
Sample Ancestral States at nodes

Ultimately, we want to annotate edges with changes of state. As a preliminary, we need to sample ancestral states at internal nodes.

We know how to calculate the most likely states at internal nodes under the joint reconstruction: we modified our product-sum algorithm for calculating the likelihood ("Felsenstein pruning") to a max-sum algorithm ("Viterbi algorithm").

Now, instead of selecting the most likely state in the pass from the root to the tips, we sample instead.

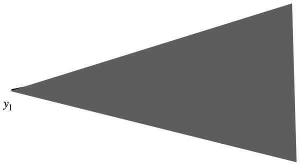
An algorithm to sample a joint reconstruction



Recall that in the pruning algorithm, we will have conditional likelihoods $L_{y_1}(z_1)$, $L_{y_1}(z_2)$, ... $L_{y_1}(z_m)$ for each of the m possible states at the root.

To obtain the most likely joint reconstruction, we would set $\hat{y}_1 = \text{argmax}_z \pi(z) L_{y_1}(z)$ as the most likely root state.

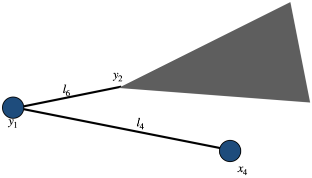
An algorithm to sample a joint reconstruction



If we want to **sample** from the distribution of joint configurations, we start by sampling the root state according to

$$Y_1 \sim \text{Multinomial}\left(\frac{\pi(z_1)L_{y_1}(z_1)}{\sum_k \pi(z_k)L_{y_1}(z_k)}, \ldots, \frac{\pi(z_m)L_{y_1}(z_m)}{\sum_k \pi(z_k)L_{y_1}(z_k)}\right)$$

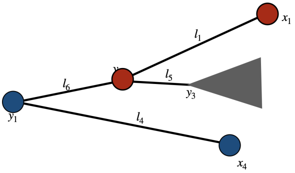
An algorithm to sample a joint reconstruction



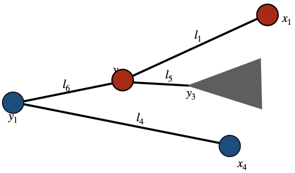
Next, we need to sample the state at y_2. To do so, we use the state we sampled at the root, call it $\tilde{y}_1$, and use $p_{\tilde{y}_1,z}(l_6)L_{y_2}(z)$ for each z (a transition probability multiplied by the likelihood of the subtree descending from y_2; this was already calculated in the tips-to-root pass).

An algorithm to sample a joint reconstruction

We sample again, but this time from the distribution
$Y_2 \sim \text{Multinomial}\left(\frac{p_{\tilde{y}_1,z_1}(l_6)L_{y_2}(z_1)}{\sum_k p_{\tilde{y}_1,z_k}L_{y_2}(z_k)}, \ldots, \frac{p_{\tilde{y}_1,z_m}(l_6)L_{y_2}(z_m)}{\sum_k p_{\tilde{y}_1,z_k}L_{y_2}(z_k)}\right)$:



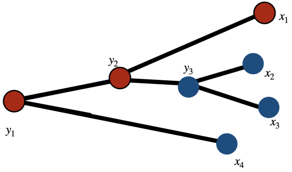
An algorithm to sample a joint reconstruction



Next, we need to sample the state at y_3. We use the state we sampled at the parent node, $\tilde{y}_2$. We use $p_{\tilde{y}_2,z}(l_5)L_{y_3}(z)$ for each z, and use these to determine probabilities in a Multinomial distribution once more.

$$Y_3 \sim \text{Multinomial}\left(\frac{p_{\tilde{y}_2,z_1}(l_5)L_{y_3}(z_1)}{\sum_k p_{\tilde{y}_2,z_k}(l_5)L_{y_3}(z_k)}, \ldots, \frac{p_{\tilde{y}_2,z_m}(l_5)L_{y_3}(z_m)}{\sum_k p_{\tilde{y}_2,z_k}(l_5)L_{y_3}(z_k)}\right)$$

An algorithm to sample a joint reconstruction

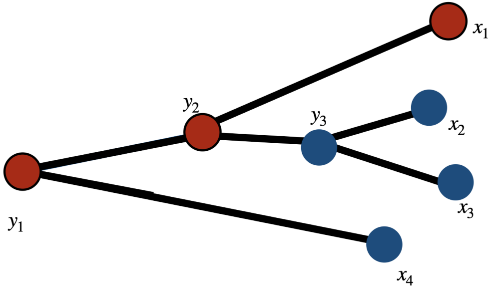


Now, we have $\tilde{\mathbf{y}} = (\tilde{y}_1, \tilde{y}_2, \tilde{y}_3)$ as our sampled reconstruction from the joint distribution. The modification from the Maximum Likelihood joint reconstruction was almost trivial – we just replace "argmax" with "sample"

Table of Contents

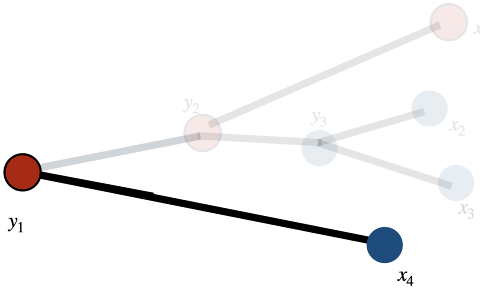
SIMULATING CTMC

Armed with a sampled collected of states at internal nodes, we proceed to simulate trait change along branches in a way that respects the states we just sampled.



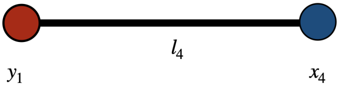
Simulating CTMC

For illustration, consider the branch connecting the root (y_1) to the tip denoted x_4.



Simulating CTMC

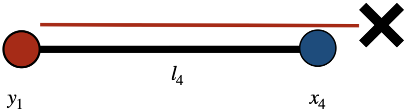
From the generator we know the rates $q_{\text{red, blue}}$ and $q_{\text{blue, red}}$.



Starting from the red state at y_1, we sample an Exponential random variable with rate $q_{\text{red, blue}}$ to simulate the time until switching into the blue state.

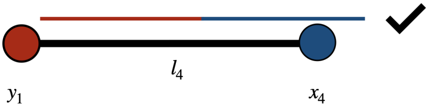
SIMULATING CTMC

If our sampled time is longer than the length of the branch (l_4), then we toss it out and try again from scratch.



SIMULATING CTMC

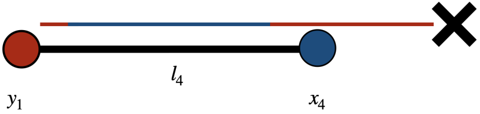
In this case, we switched into the blue state before time l_4. Then, we simulate another Exponential random variable, but this time with rate $q_{\text{blue, red}}$ to simulate the time until switching from blue back to red.



This particular simulation is compatible with the node states.

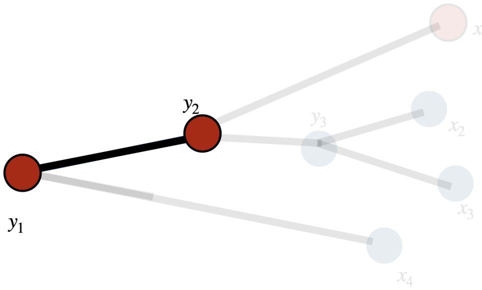
Simulating CTMC

This one is not compatible:



Simulating CTMC

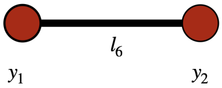
As one more example, let's consider the edge connecting the root, y_1, to the internal node y_2:



We simulate different edges independently of each other (conditional on the sampled states at the internal nodes).

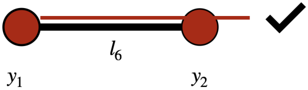
Simulating CTMC

We need to start and end in the red state:



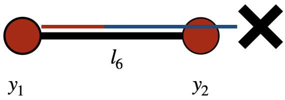
Simulating CTMC

This could happen if we never move out of the red state before time l_6:



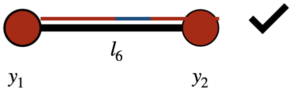
Simulating CTMC

This one doesn't work:



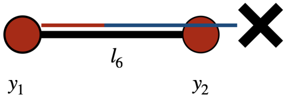
Simulating CTMC

This one works:



Discarding simulations

When we simulate trajectories that fail to satisfy our boundary conditions on edges, like this one,



it's important to recognize that we have to restart from state y_1 every time. Otherwise, severe biases are introduced

Table of Contents

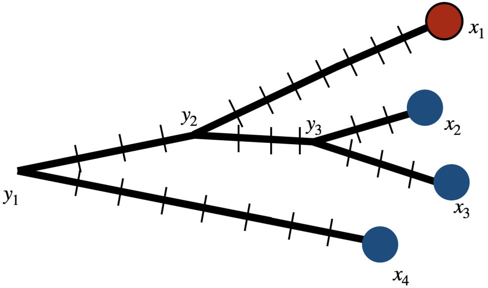
A more generalizable approach: discretization

There's another approach to this that generalizes to more complicated models.

Instead of simulating character histories, we just discretize our edges and then apply forward/backward passes to sample states. We know how to sample states at internal nodes of our phylogeny; the same approach works just fine at discretization points along edges.

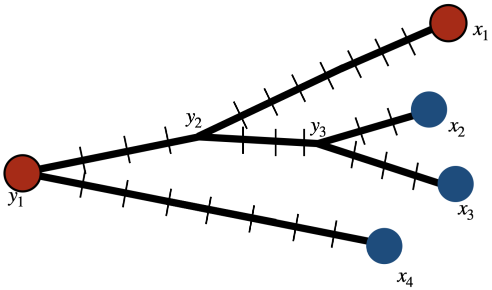
Character mapping from discretization

First, impose a discretization along the edges that respects the internal nodes:



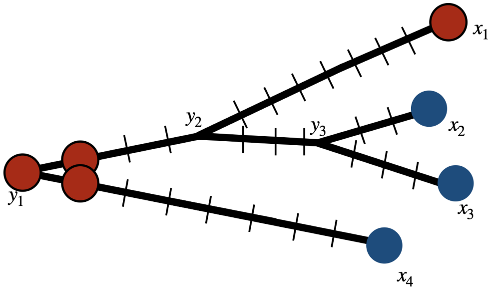
Character mapping from discretization

Sample a state at the root:

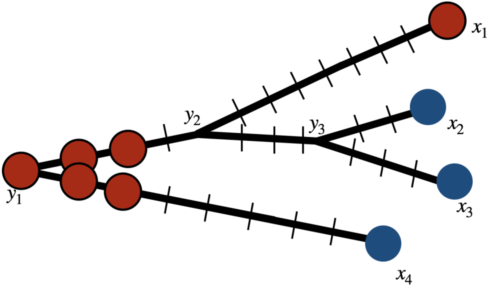


Character mapping from discretization

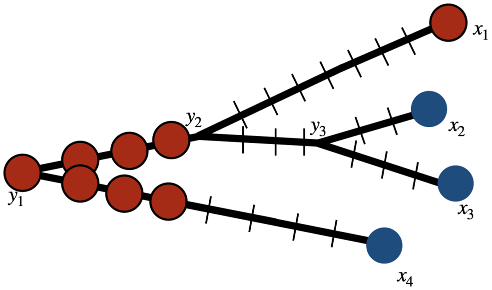
Then, proceed from the root to the tips, sampling states from Multinomial distributions at each step:



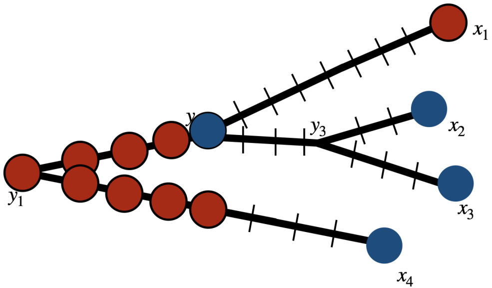
Character mapping from discretization



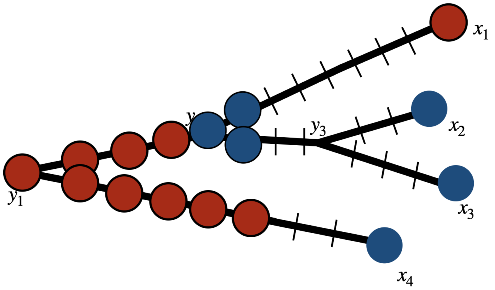
Character mapping from discretization



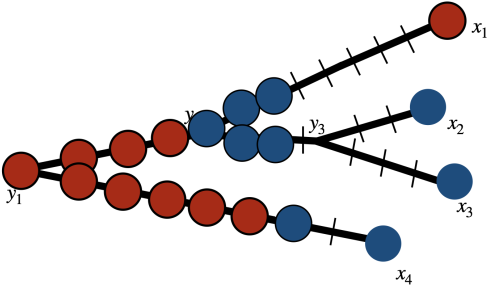
Character mapping from discretization



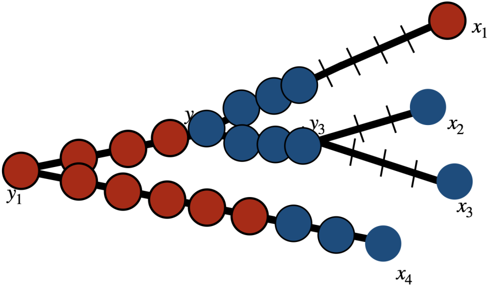
Character mapping from discretization



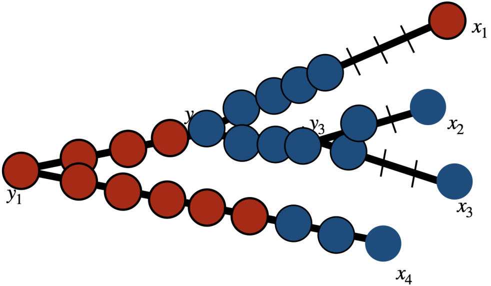
Character mapping from discretization



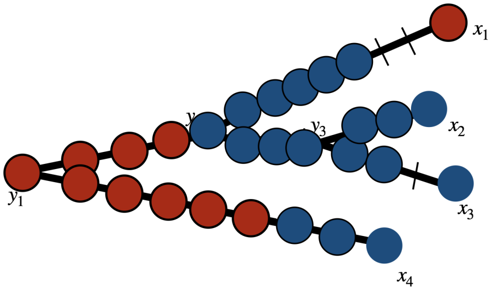
Character mapping from discretization



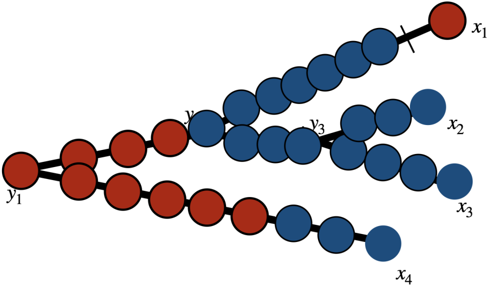
Character mapping from discretization



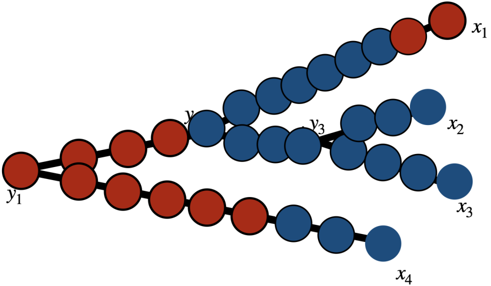
Character mapping from discretization



Character mapping from discretization

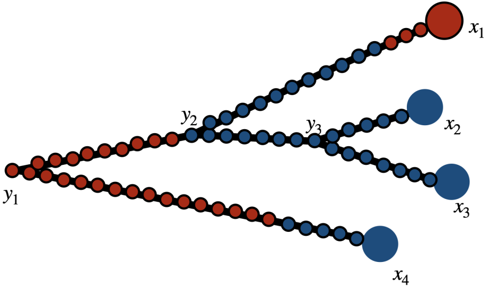


Character mapping from discretization



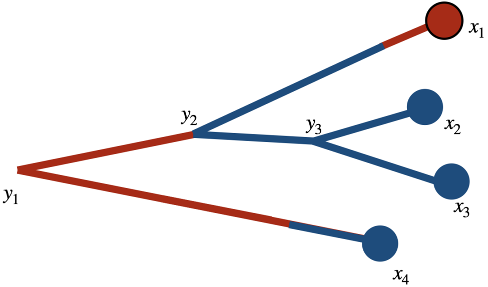
Character mapping from discretization

In the limit as we add infinitely many discretization points...



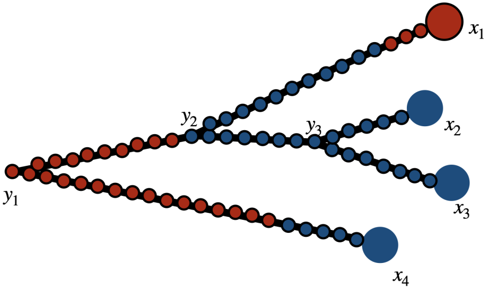
Character mapping from discretization

... we recover the same character mappings we would obtain by simulating CTMC in continuous time.



In addition: note that this method allows us to obtain the most likely character mapping, which the continuous-time simulation approach does not provide

Character mapping from discretization



With this approach, we never toss out continuous-time simulations that fail to be consistent with boundary conditions.